arXiv:2606.19348v1 [cs.CL] 26 Apr 2026

DeepSeek-V4:
Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

Abstract

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (*m*HC) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at `https://huggingface.co/collections/deepseek-ai/deepseek-v4`.

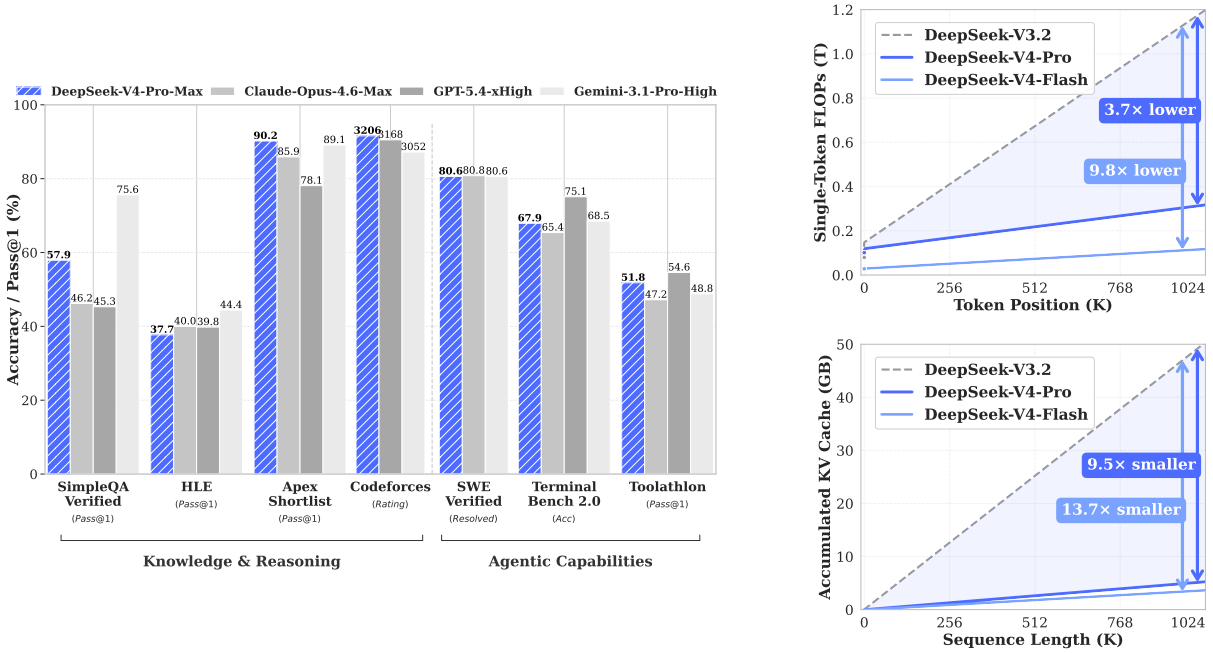


Figure 1 | **Left**: benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. **Right**: inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

Contents

1 Introduction **4**

2 Architecture **6**

2.1 Designs Inherited from DeepSeek-V3 7

2.2 Manifold-Constrained Hyper-Connections 7

2.3 Hybrid Attention with CSA and HCA 9

2.3.1 Compressed Sparse Attention 9

2.3.2 Heavily Compressed Attention 11

2.3.3 Other Details 12

2.3.4 Efficiency Discussion 13

2.4 Muon Optimizer 14

3 General Infrastructures **15**

3.1 Fine-Grained Communication-Computation Overlap in Expert Parallelism 15

3.2 Flexible and Efficient Kernel Development with TileLang 16

3.3 High-Performance Batch-Invariant and Deterministic Kernel Libraries 18

3.4 Training Framework 19

3.4.1 Efficient Implementation of Muon 19

3.4.2 Cost-Effective and Memory-Efficient Implementation of mHC 20

3.4.3 Contextual Parallelism for Long-Context Attention 20

3.4.4 Extended Automatic Differentiation for Flexible Activation Checkpointing 21

3.5 Inference Framework 21

3.5.1 KV Cache Structure and Management 21

3.5.2 On-Disk KV Cache Storage 23

4 Pre-Training **24**

4.1 Data Construction 24

4.2 Pre-Training Setups 24

4.2.1 Model Setups 24

4.2.2 Training Setups 25

4.2.3 Mitigating Training Instability 26

4.3 Evaluations 27

4.3.1 Evaluation Benchmarks 27

4.3.2 Evaluation Results 27

5 Post-Training **28**
5.1 Post-Training Pipeline . . . 28
5.1.1 Specialist Training . . . 28
5.1.2 On-Policy Distillation . . . 32
5.2 Post-Training Infrastructures . . . 33
5.2.1 FP4 Quantization-Aware Training . . . 33
5.2.2 Efficient Teacher Scheduling for Full-Vocabulary OPD . . . 34
5.2.3 Preemptible and Fault-Tolerant Rollout Service . . . 34
5.2.4 Scaling RL Framework for Million-Token Context . . . 35
5.2.5 Sandbox Infrastructure for Agentic AI . . . 35
5.3 Standard Benchmark Evaluation . . . 36
5.3.1 Evaluation Setup . . . 36
5.3.2 Evaluation Results . . . 37
5.4 Performance on Real-World Tasks . . . 41
5.4.1 Chinese Writing . . . 41
5.4.2 Search . . . 42
5.4.3 White-Collar Task . . . 42
5.4.4 Code Agent . . . 44

6 Conclusion, Limitations, and Future Directions **44**

A Author List and Acknowledgment **54**
A.1 Author List . . . 54
A.2 Acknowledgment . . . 55

B Evaluation Details **55**

1. Introduction

The emergence of reasoning models (DeepSeek-AI, 2025; OpenAI, 2024c) has established a new paradigm of test-time scaling, driving substantial performance gains for Large Language Models (LLMs). However, this scaling paradigm is fundamentally constrained by the quadratic computational complexity of the vanilla attention mechanism (Vaswani et al., 2017), which creates a prohibitive bottleneck for ultra-long contexts and reasoning processes. Concurrently, the emergence of long-horizon scenarios and tasks — from complex agentic workflows to massive cross-document analysis — has also made efficient support for ultra-long contexts critical for future progress. While recent open-source efforts (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) have advanced general capabilities, this core architectural inefficiency in handling ultra-long sequences remains a key impediment, limiting further gains from test-time scaling and hindering further exploration into long-horizon scenarios and tasks.

In order to break the efficiency barrier in ultra-long contexts, we develop the DeepSeek-V4 series, including the preview versions of DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated). Through architectural innovations, DeepSeek-V4 series achieve a dramatic leap in computational efficiency for processing ultra-long sequences. This breakthrough enables efficient support for a context length of one million tokens, ushering in a new era of million-length contexts for next-generation LLMs. We believe our capability to efficiently handle ultra-long sequences unlocks the next frontier of test-time scaling, paves the way for deeper research into long-horizon tasks, and establishes a necessary foundation for exploring future paradigms like online learning.

Compared with the DeepSeek-V3 architecture (DeepSeek-AI, 2024), DeepSeek-V4 series retain the DeepSeekMoE framework (Dai et al., 2024) and Multi-Token Prediction (MTP) strategy, while introducing several key innovations in architecture and optimization. To enhance long-context efficiency, we design a hybrid attention mechanism combining Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA). CSA compresses the KV caches along the sequence dimension and then performs DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025), whereas HCA applies more aggressive compression to the KV caches but keeps dense attention. To strengthen modeling capability, we incorporate Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) that upgrade conventional residual connections. Additionally, we introduce the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer to the training of DeepSeek-V4 series, leading to faster convergence and improved training stability.

To enable efficient training and inference for DeepSeek-V4 series as well as productive development, we introduce several infrastructure optimizations. First, we design and implement a single fused kernel for MoE modules that fully overlaps computation, communication, and memory access. Second, we employ TileLang (Wang et al., 2026), a Domain-Specific Language (DSL) to balance development productivity and runtime efficiency. Third, we provide efficient batch-invariant and deterministic kernel libraries to ensure bitwise reproducibility across training and inference. Fourth, for the training framework, we extend the autograd framework with tensor-level checkpointing for fine-grained recomputation control; and we enhance training efficiency with a hybrid ZeRO strategy for the Muon optimizer, cost-effective mHC implementations via recomputation and fused kernels, and two-stage contextual parallelism to manage compressed attention. Fifth, for the inference framework, we design a heterogeneous KV cache structure with on-disk storage strategies to enable efficient shared-prefix reuse. In addition, during the post-training stage, we incorporate FP4 quantization-aware training for MoE expert weights and the indexer QK path to reduce memory and computation.

By employing hybrid CSA and HCA, along with precision optimizations on computation and storage, DeepSeek-V4 series achieve significantly lower inference FLOPs and a substantially reduced KV cache size compared with DeepSeek-V3.2, especially in long-context settings. The right part of Figure 1 demonstrates the estimated single-token inference FLOPs and accumulated KV cache size of DeepSeek-V3.2 and DeepSeek-V4 series. In the scenario of 1M-token context, even DeepSeek-V4-Pro, which has a larger number of activated parameters, attains only 27% of the single-token FLOPs (measured in equivalent FP8 FLOPs) and 10% of the KV cache size relative to DeepSeek-V3.2. Furthermore, DeepSeek-V4-Flash, with its smaller number of activated parameters, pushes efficiency even further: in the 1M-token context setting, it achieves only 10% of the single-token FLOPs and 7% of the KV cache size compared with DeepSeek-V3.2. Additionally, for DeepSeek-V4 series, the routed expert parameters utilize FP4 precision. While the peak FLOPs for FP4 $\times$ FP8 operations are currently the same as FP8 $\times$ FP8 on existing hardware, they can theoretically be implemented to be 1/3 more efficient on future hardware, which will further enhance the efficiency of DeepSeek-V4 series.

During pre-training, we train DeepSeek-V4-Flash on 32T tokens and DeepSeek-V4-Pro on 33T tokens, respectively. After pre-training, these two models can natively and efficiently support 1M-length contexts. In our internal evaluations, DeepSeek-V4-Flash-Base already surpasses DeepSeek-V3.2-Base across a majority of benchmarks with its more parameter-efficient design. DeepSeek-V4-Pro-Base further extends this advantage to set a new performance standard among DeepSeek foundation models, achieving comprehensive superiority across reasoning, coding, long-context, and world knowledge tasks.

The post-training pipeline of DeepSeek-V4 series features a two-stage paradigm: the independent cultivation of domain-specific experts, followed by unified model consolidation via on-policy distillation (Gu et al., 2024; Lu and Lab, 2025). Initially, for each target domain — such as mathematics, coding, agent, and instruction following — a separate expert model is trained independently. The base model first undergoes Supervised Fine-Tuning (SFT) on high-quality, domain-specific data to establish foundational capabilities. Subsequently, Reinforcement Learning (RL) is applied using Group Relative Policy Optimization (GRPO) (DeepSeek-AI, 2025), which further optimizes the model for domain-aligned behaviors guided by reward models tailored to specific success criteria. This phase yields a diverse set of specialized experts, each excelling in its respective field. Finally, to integrate these distinct proficiencies, a single unified model is trained through on-policy distillation, wherein the unified model acts as the student learning to optimize the reverse KL loss with teacher models.

Summary of Core Evaluation Results

- **Knowledge**: In assessments of broad world knowledge, DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, significantly outperforms leading open-source models on the SimpleQA (OpenAI, 2024d) and Chinese-SimpleQA (He et al., 2024) benchmarks. Regarding educational knowledge — evaluated via MMLU-Pro (Wang et al., 2024b), HLE (Phan et al., 2025), and GPQA (Rein et al., 2023) — DeepSeek-V4-Pro-Max shows a marginal lead over its open-source counterparts. DeepSeek-V4-Pro-Max has significantly closed the gap with the leading proprietary model, Gemini-3.1-Pro, despite still trailing it in these knowledge-based evaluations.
- **Reasoning**: Through the expansion of reasoning tokens, DeepSeek-V4-Pro-Max demonstrates superior performance relative to GPT-5.2 and Gemini-3.0-Pro on standard reasoning benchmarks. Nevertheless, its performance falls marginally short of GPT-5.4 and Gemini-3.1-Pro, suggesting a developmental trajectory that trails state-of-the-art frontier models by approximately 3 to 6 months. Furthermore, DeepSeek-V4-Flash-Max achieves comparable

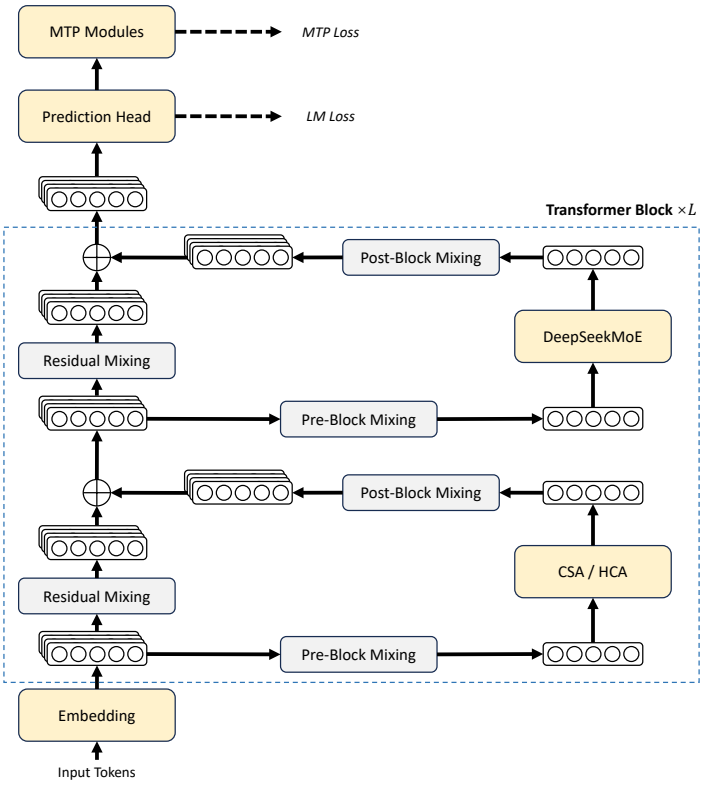


Figure 2 | Overall architecture of DeepSeek-V4 series. We use hybrid CSA (Compressed Sparse Attention) and HCA (Heavily Compressed Attention) for attention layers, DeepSeekMoE for feed-forward layers, and strengthen conventional residual connections with mHC.

performance to GPT-5.2 and Gemini-3.0-Pro, establishing itself as a highly cost-effective architecture for complex reasoning tasks.

- **Agent**: On public benchmarks, DeepSeek-V4-Pro-Max is on par with leading open-source models, such as Kimi-K2.6 and GLM-5.1, but slightly worse than frontier closed models. In our internal evaluation, DeepSeek-V4-Pro-Max outperforms Claude Sonnet 4.5 and approaches the level of Opus 4.5.
- **Long-Context**: DeepSeek-V4-Pro-Max delivers strong results on synthetic and real use cases with a 1-million-token context window, surpassing even Gemini-3.1-Pro on academic benchmarks.
- **DeepSeek-V4-Pro v.s. DeepSeek-V4-Flash**: DeepSeek-V4-Flash-Max exhibits lower performance in knowledge evaluations due to its smaller parameter scale. However, it achieves comparable results on reasoning tasks when allocated a larger thinking budget. In agent evaluations, while DeepSeek-V4-Flash-Max matches the performance of DeepSeek-V4-Pro-Max on several benchmarks, it still trails its larger counterpart on more complex, high-difficulty tasks.

2. Architecture

Overall, DeepSeek-V4 series retain the Transformer (Vaswani et al., 2017) architecture and Multi-Token Prediction (MTP) modules (DeepSeek-AI, 2024; Gloeckle et al., 2024), while introducing several key upgrades over DeepSeek-V3: (1) firstly, we introduce the Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) to strengthen conventional residual connections;

(2) secondly, we design a hybrid attention architecture, which greatly improves long-context efficiency through Compressed Sparse Attention and Heavily Compressed Attention. (3) thirdly, we employ Muon (Jordan et al., 2024; Liu et al., 2025) as the optimizer. For the Mixture-of-Experts (MoE) components, we still adopt the DeepSeekMoE (Dai et al., 2024) architecture, with only minor adjustments from DeepSeek-V3. The Multi-Token Prediction (MTP) (DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024; Qi et al., 2020) configuration remains identical to that of DeepSeek-V3. All other unspecified details follow the settings established in DeepSeek-V3 (DeepSeek-AI, 2024). Figure 2 illustrates the overall architecture of DeepSeek-V4, and the details are described below.

2.1. Designs Inherited from DeepSeek-V3

Mixture-of-Experts. As previous DeepSeek-series models (DeepSeek-AI, 2024; DeepSeek-AI, 2024), DeepSeek-V4 series also adopt the DeepSeekMoE paradigm (Dai et al., 2024) for Feed-Forward Networks (FFNs), which sets fine-grained routed experts and shared experts. Different from DeepSeek-V3, we change the activation function that computes the affinity scores from Sigmoid$(\cdot)$ into Sqrt(Softplus$(\cdot)$). For load balancing, we also employ the auxiliary-loss-free strategy (DeepSeek-AI, 2024; Wang et al., 2024a), augmented by a slight sequence-wise balance loss that prevents extreme imbalance within individual sequences. For DeepSeek-V4, we remove the constraint on the number of routing target nodes, and carefully redesign the parallelism strategy to maintain training efficiency. Furthermore, compared with DeepSeek-V3, we replace the dense FFN layers in the initial several Transformer blocks with MoE layers that employ Hash routing (Roller et al., 2021). The Hash routing strategy determines the target experts of each token according to a predefined hash function with regard to the input token ID.

Multi-Token Prediction. As DeepSeek-V3, DeepSeek-V4 series also set MTP modules and objectives. Given that the MTP strategy has been validated in DeepSeek-V3, we adopt the same strategy for DeepSeek-V4 series without modification.

2.2. Manifold-Constrained Hyper-Connections

As shown in Figure 2, DeepSeek-V4 series incorporate Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) to strengthen the conventional residual connections between adjacent Transformer blocks. Compared with naive Hyper-Connections (HC) (Zhu et al., 2025), the core idea of mHC is to constrain the residual mapping onto a specific manifold, and thus enhance the stability of signal propagation across layers while preserving model expressivity. This subsection briefly introduces the standard HC and describes how we design mHC for stable training.

Standard Hyper-Connections. The standard HC expands the width of the residual stream by a factor of n_{hc}. Specifically, the shape of the residual stream is expanded from $\mathbb{R}^d$ to $\mathbb{R}^{n_{\text{hc}} \times d}$, where d is the hidden size of the actual layer input. Let $X_l = [\mathbf{x}_{l,1}; \ldots; \mathbf{x}_{l,n_{\text{hc}}}]^T \in \mathbb{R}^{n_{\text{hc}} \times d}$ be the residual state before the l-th layer. HC introduces three linear mappings: an input mapping $A_l \in \mathbb{R}^{1 \times n_{\text{hc}}}$, a residual transformation $B_l \in \mathbb{R}^{n_{\text{hc}} \times n_{\text{hc}}}$, and an output mapping $C_l \in \mathbb{R}^{n_{\text{hc}} \times 1}$. The update of the residual state is then formulated as:

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \tag{1}$$

where $\mathcal{F}_l$ denotes the l-th layer (e.g., an MoE layer), whose input and output shapes are both $\mathbb{R}^d$. Note that the actual layer input $A_l X_l \in \mathbb{R}^d$ is also d-dimensional, so the expanded residual

width does not influence the design of the inner layers. HC decouples the residual width from the actual hidden size, offering a complementary scaling axis with minimal computational overhead, as n_{hc} is typically much smaller than the hidden size d. However, even though HC has demonstrated potential in improving model performance, we find that the training will frequently exhibit numerical instability when stacking multiple layers, which hinders the scaling of HC.

Manifold-Constrained Residual Mapping. The core innovation of mHC is to constrain the residual mapping matrix B_l to the manifold of doubly stochastic matrices (the Birkhoff polytope) $\mathcal{M}$, and thus enhance the stability of signal propagation across layers:

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n\times n} \mid M\mathbf{1}_n = \mathbf{1}_n,\ \mathbf{1}_n^T M = \mathbf{1}_n^T,\ M \geqslant 0\}. \tag{2}$$

This constraint ensures that the spectral norm of the mapping matrix $\|B_l\|_2$ is bounded by 1, so the residual transformation is non-expansive, which increases the numerical stability during both the forward pass and backpropagation. Besides, the set $\mathcal{M}$ is closed under multiplication, which guarantees stability in the scenarios of deep stacks of mHC. In addition, the input transformation A_l and output transformation C_l are also constrained to be non-negative and bounded via a Sigmoid function to avoid the risk of signal cancellation.

Dynamic Parameterization. The parameters of three linear mappings are dynamically generated, which are decomposed into a dynamic (input-dependent) component and a static (input-independent) component. Given the input $X_l \in \mathbb{R}^{n_{\text{hc}}\times d}$, it is first flattened and normalized: $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1\times n_{\text{hc}}d}$. Then, we follow the conventional HC to generate the unconstrained raw parameters $\tilde{A}_l \in \mathbb{R}^{1\times n_{\text{hc}}}$, $\tilde{B}_l \in \mathbb{R}^{n_{\text{hc}}\times n_{\text{hc}}}$, and $\tilde{C}_l \in \mathbb{R}^{n_{\text{hc}}\times 1}$:

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \tag{3}$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \tag{4}$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \tag{5}$$

where $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}}$ and $W_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}d\times n_{\text{hc}}^2}$ are learnable parameters for generating the dynamic components; $\text{Mat}(\cdot)$ reshapes a vector of size $1\times n_{\text{hc}}^2$ into a matrix of size $n_{\text{hc}}\times n_{\text{hc}}$; $S_l^{\text{pre}} \in \mathbb{R}^{1\times n_{\text{hc}}}$, $S_l^{\text{post}} \in \mathbb{R}^{n_{\text{hc}}\times 1}$, and $S_l^{\text{res}} \in \mathbb{R}^{n_{\text{hc}}\times n_{\text{hc}}}$ are learnable static biases; and $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}} \in \mathbb{R}$ are learnable gating factors initialized to small values.

Applying Parameter Constraints. After obtaining the unconstrained raw parameters $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$, we then apply constraints described earlier to them to enhance the numerical stability. To be specific, for the input and output mappings, we employ a Sigmoid function $\sigma(\cdot)$ to ensure their non-negativity and boundedness:

$$A_l = \sigma(\tilde{A}_l), \tag{6}$$

$$C_l = 2\sigma(\tilde{C}_l). \tag{7}$$

As for the residual mapping $\tilde{B}_l$, we project it onto the manifold of doubly stochastic matrices $\mathcal{M}$. This is achieved by the Sinkhorn-Knopp algorithm, which first applies an exponential function to $\tilde{B}_l$ to ensure positivity, getting $M^{(0)} = \exp(\tilde{B}_l)$, and then iteratively performs column and row normalization:

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})), \tag{8}$$

where $\mathcal{T}_r$ and $\mathcal{T}_c$ denote row and column normalization, respectively. This iteration converges to a constrained doubly stochastic matrix $B_l = M^{(t_{\max})}$. We choose $t_{\max} = 20$ as a practical value.

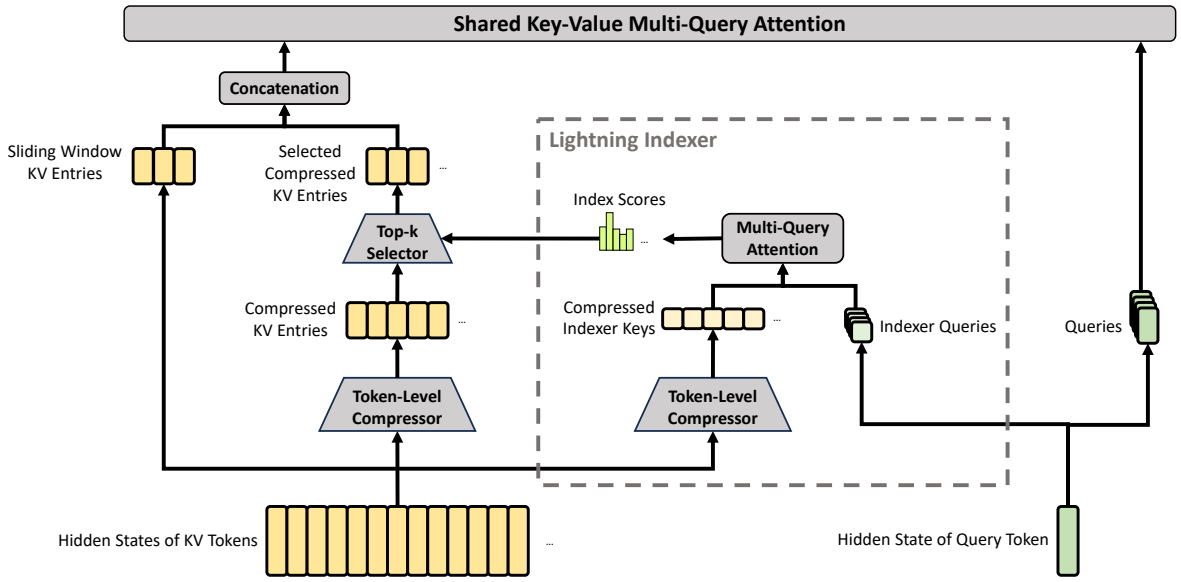


Figure 3 | Core architectures of CSA. It compresses the number of KV entries to $\frac{1}{m}$ times, and then applies DeepSeek Sparse Attention for further acceleration. Additionally, a small set of sliding window KV entries is combined with the selected compressed KV entries to enhance local fine-grained dependencies.

2.3. Hybrid Attention with CSA and HCA

As the context length reaches extreme scales, the attention mechanism emerges as the dominant computational bottleneck in a model. For DeepSeek-V4, we design two efficient attention architectures — Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) — and employ their interleaved hybrid configuration, which substantially reduces the computational cost of attention in long-text scenarios. CSA integrates both compression and sparse attention strategies: it first compresses the Key-Value (KV) cache of every m tokens into one entry, and then applies DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025) where each query token attends to only k compressed KV entries. HCA aims for extreme compression by consolidating the KV cache of every m' ($\gg m$) tokens into a single entry. The hybrid architecture of CSA and HCA remarkably improves the long-context efficiency of DeepSeek-V4 series, making one-million-token context feasible in practice. This subsection describes the core techniques of our hybrid attention architecture, and we also provide an open-source implementation[1] to specify more details unambiguously.

2.3.1. Compressed Sparse Attention

The core architecture of CSA is illustrated in Figure 3, which first compresses the KV cache of each m tokens into one entry, and then applies DeepSeek Sparse Attention for further acceleration.

Compressed Key-Value Entries. Let $H \in \mathbb{R}^{n \times d}$ be a sequence of input hidden states, where n is the sequence length and d is the hidden size. CSA first computes two series of KV entries $C^a, C^b \in \mathbb{R}^{n \times c}$ and their corresponding compression weights $Z^a, Z^b \in \mathbb{R}^{n \times c}$, where c is the head

[1] `https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference`

dimension:

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \tag{9}$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \tag{10}$$

where $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in \mathbb{R}^{d \times c}$ are trainable parameters. Next, each m KV entries in C^a and C^b will be compressed into one entry according to their compression weights and learnable positional biases $B^a, B^b \in \mathbb{R}^{m \times c}$, producing $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m} \times c}$. Each compressed entry $C_i^{\text{Comp}} \in \mathbb{R}^c$ is computed by

$$[S^a_{mi:m(i+1)-1}; S^b_{m(i-1):mi-1}] = \text{Softmax}_{\text{row}}([Z^a_{mi:m(i+1)-1} + B^a; Z^b_{m(i-1):mi-1} + B^b]), \tag{11}$$

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b, \tag{12}$$

where $\odot$ denotes the Hadamard product; $\text{Softmax}_{\text{row}}(\cdot)$ denotes the softmax operation along the row dimension, which performs normalization across the total of $2m$ elements from both Z^a and Z^b. When $i = 0$, $Z^b_{m(i-1):mi-1}$ is padded with negative infinity and $C^b_{m(i-1):mi-1}$ is padded with zeros. Note that each C_i^{Comp} is derived from $2m$ KV entries, but the indexes of C^b used for C_i^{Comp} and the indexes of C^a used for C_{i-1}^{Comp} are overlapped. Therefore, CSA in fact compresses the sequence length to $\frac{1}{m}$ times.

Lightning Indexer for Sparse Selection. After obtaining the compressed KV entries C^{Comp}, CSA applies the DSA strategy to select top-k compressed KV entries for core attention. First, CSA performs the same compression operation used for C^{Comp} to get compressed indexer keys $K^{\text{IComp}} \in \mathbb{R}^{\frac{n}{m} \times c^I}$, where c^I is the indexer head dimension. Then, for a query token t, we produce the indexer queries $\{\mathbf{q}^I_{t,1}; \mathbf{q}^I_{t,2}; ...; \mathbf{q}^I_{t,n^I_h}\}$ in a low-rank manner:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \tag{13}$$

$$[\mathbf{q}^I_{t,1}; \mathbf{q}^I_{t,2}; ...; \mathbf{q}^I_{t,n^I_h}] = \mathbf{q}^I_t = \mathbf{c}_t^Q \cdot W^{IUQ}, \tag{14}$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the input hidden state of the query token t; $\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ is the compressed latent vector for queries; d_c denotes the query compression dimension; n^I_h denotes the number of indexer query heads; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ and $W^{IUQ} \in \mathbb{R}^{d_c \times c^I n^I_h}$ are the down-projection and up-projection matrices for indexer queries, respectively. Next, the index score $I_{t,s} \in \mathbb{R}$ between the query token t and a preceding compressed block s ($s < \text{Floor}(\frac{t}{m})$) is computed by

$$[w^I_{t,1}; w^I_{t,2}; ...; w^I_{t,n^I_h}] = \mathbf{w}^I_t = \mathbf{h}_t \cdot W^w, \tag{15}$$

$$I_{t,s} = \sum_{h=1}^{n^I_h} w^I_{t,h} \cdot \text{ReLU}\left(\mathbf{q}^I_{t,h} \cdot K^{\text{IComp}}_s\right), \tag{16}$$

where $W^w \in \mathbb{R}^{d \times n^I_h}$ is a learnable matrix; $w^I_{t,h} \in \mathbb{R}$ is the weight of the h-th indexer head. For a query token t, given its index scores $I_{t,:}$, we employ a top-k selector to selectively retain a subset of compressed KV entries C_t^{SprsComp} for subsequent core attention:

$$C_t^{\text{SprsComp}} = \left\{ C_s^{\text{Comp}} \,\middle|\, I_{t,s} \in \text{Top-k}(I_{t,:}) \right\}. \tag{17}$$

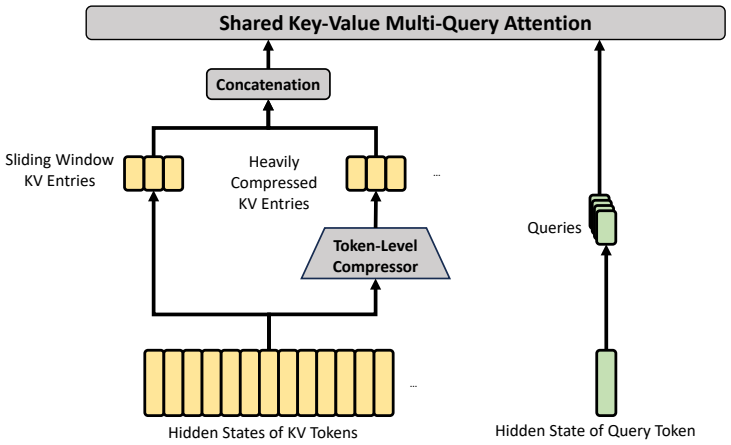


Figure 4 | Core architectures of HCA. It performs heavier compression, where the KV entries of m' ($\gg m$) tokens will be consolidated into one. Also, we additionally introduce a small set of sliding window KV entries to enhance local fine-grained dependencies.

Shared Key-Value MQA. After selecting the sparse KV entries, CSA then performs core attention in a Multi-Query Attention (MQA) (Shazeer, 2019) manner, where each compressed KV entry in C_t^{SprsComp} serves as both attention key and value. To be specific, for a query token t, we first produce attention queries $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}\}$ from the compressed latent vector $\mathbf{c}_t^Q$:

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \tag{18}$$

where n_h denotes the number of query heads; $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ is the up-projection matrices for queries. Note that the latent query vector $\mathbf{c}_t^Q$ is shared with that used for the indexer queries. Next, we perform MQA on $\{\mathbf{q}_{t,i}\}$ and C_t^{SprsComp}:

$$\mathbf{o}_{t,i} = \text{CoreAttn}\left(\texttt{query=}\mathbf{q}_{t,i}, \texttt{key=}C_t^{\text{SprsComp}}, \texttt{value=}C_t^{\text{SprsComp}}\right), \tag{19}$$

where $\mathbf{o}_{t,i} \in \mathbb{R}^c$ is the core attention output of the i-th head at the t-th token; $\text{CoreAttn}(\cdot)$ denotes the core attention operation.

Grouped Output Projection. In the configuration of DeepSeek-V4, cn_h is quite large. Therefore, directly projecting the outputs of the core attention operation $[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; ...; \mathbf{o}_{t,n_h}] = \mathbf{o}_t \in \mathbb{R}^{cn_h}$ to a d-dimensional hidden state will impose a substantial computational burden. To mitigate this cost, we design a grouped output projection strategy. To be specific, we first split n_h outputs into g groups, and then for each group of output $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c\frac{n_h}{g}}$, we project it to a d_g-dimensional intermediate output $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$, where $d_g < c\frac{n_h}{g}$. Finally, we project the intermediate output $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; ...; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ to the final attention output $\mathbf{\hat{o}}_t \in \mathbb{R}^d$.

2.3.2. Heavily Compressed Attention

The core architecture of HCA is illustrated in Figure 4, which compresses the KV cache in a heavier manner, but does not employ sparse attention.

Compressed Key-Value Entries. By and large, the compression strategy of HCA is similar to that of CSA, but employs a larger compression rate m' ($\gg m$) and does not perform overlapped

compression. Let $H \in \mathbb{R}^{n \times d}$ be a sequence of input hidden states, HCA first computes the original KV entries $C \in \mathbb{R}^{n \times c}$ and their corresponding compression weights $Z \in \mathbb{R}^{n \times c}$:

$$C = H \cdot W^{KV}, \tag{20}$$

$$Z = H \cdot W^{Z}, \tag{21}$$

where $W^{KV}, W^{Z} \in \mathbb{R}^{d \times c}$ are trainable parameters. Next, each m' KV entries in C will be compressed into one according to the compression weights and learnable positional biases $B \in \mathbb{R}^{m' \times c}$, producing $C^{\text{Comp}} \in \mathbb{R}^{\frac{n}{m'} \times c}$. Each compressed entry $C_i^{\text{Comp}} \in \mathbb{R}^c$ is computed by

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \tag{22}$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j. \tag{23}$$

Through this compression operation, HCA compresses the sequence length to $\frac{1}{m'}$ times.

Shared Key-Value MQA and Grouped Output Projection. HCA also employs the shared KV MQA and grouped output projection strategies as CSA does. After the KV compression, for a query token t, HCA first produces attention queries $\{\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}\}$ in a low-rank manner:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \tag{24}$$

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; ...; \mathbf{q}_{t,n_h}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \tag{25}$$

where $\mathbf{h}_t \in \mathbb{R}^d$ is the input hidden state of the query token t; n_h denotes the number of query heads; $W^{DQ} \in \mathbb{R}^{d \times d_c}$ and $W^{UQ} \in \mathbb{R}^{d_c \times cn_h}$ are the down-projection and up-projection matrices for queries, respectively. Next, we perform MQA on $\{\mathbf{q}_{t,i}\}$ and C^{Comp}:

$$\mathbf{o}_{t,i} = \text{CoreAttn}\left(\texttt{query=}\mathbf{q}_{t,i}, \texttt{key=}C^{\text{Comp}}, \texttt{value=}C^{\text{Comp}}\right), \tag{26}$$

where $\mathbf{o}_{t,i} \in \mathbb{R}^c$ is the core attention output of the i-th head at the t-th token. Next, as CSA does, HCA splits n_h outputs into g groups, and for each group of output $\mathbf{o}_{t,i}^G \in \mathbb{R}^{c\frac{n_h}{g}}$, HCA projects it to a d_g-dimensional intermediate output $\mathbf{o}_{t,i}^{G'} \in \mathbb{R}^{d_g}$, where $d_g < c\frac{n_h}{g}$. Finally, HCA projects the intermediate output $[\mathbf{o}_{t,1}^{G'}; \mathbf{o}_{t,2}^{G'}; ...; \mathbf{o}_{t,g}^{G'}] \in \mathbb{R}^{d_g g}$ to the final attention output $\mathbf{\hat{o}}_t \in \mathbb{R}^d$.

2.3.3. Other Details

In addition to the core architectures of CSA and HCA described above, our hybrid attention incorporates several other techniques. For writing clarity, we omit these additional techniques from the above introduction and will briefly describe them in this subsection. Also, this subsection focuses only on the core ideas of them and may omit some tiny details for simplicity. We encourage the readers to refer to our open-source implementation for unambiguous details.

Query and Key-Value Entry Normalization. For both CSA and HCA, we perform an additional RMSNorm operation on each head of the queries and the only head of the compressed KV entries, just before the core attention operation. This normalization avoids exploding attention logits and may improve training stability.

Partial Rotary Positional Embedding. For both CSA and HCA, we partially employ the Rotary Positional Embedding (RoPE) (Su et al., 2024) to the attention queries, KV entries, and the core attention outputs. To be specific, for each query vector and KV entry vector used in CSA and HCA, we apply RoPE to its last 64 dimensions. Since the KV entries serve as both attention keys and values, the naive core attention outputs $\{\mathbf{o}_{t,i}\}$ will carry absolute position embeddings, derived from the weighted sum of KV entries. As a countermeasure, we also apply RoPE with position $-i$ on the last 64 dimensions of each $\mathbf{o}_{t,i}$. In this way, the output of the core attention will also carry relative position embeddings — the contribution of each KV entry to the core attention outputs will also be related to the distance between the query and the KV entry.

Additional Branch of Sliding Window Attention. In order to strictly preserve causality in CSA and HCA, each query attends to only preceding compressed KV blocks. Consequently, a query cannot access information from other tokens within its own compressed block. Meanwhile, recent tokens usually possess greater relevance to the query token in language modeling. For these reasons, we introduce a supplementary attention branch to both CSA and HCA in a sliding window manner, for better modeling of local dependencies. To be specific, for each query token, we additionally produce n_{win} uncompressed KV entries corresponding to the recent n_{win} tokens. In the core attention of CSA and HCA, these KV entries in the sliding window will be used along with the compressed KV entries.

Attention Sink. In the core attention of CSA and HCA, we employ the trick of attention sink (OpenAI, 2025; Xiao et al., 2024). To be specific, we set a series of learnable sink logits $\{z'_1, z'_2, ..., z'_{n_h}\}$. For the h-th attention head, $\text{Exp}(z'_h)$ will be added to the denominator of the attention score:

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \tag{27}$$

where $s_{h,i,j}, z_{h,i,j} \in \mathbb{R}$ denote the attention score and attention logit of the h-th attention head between the i-th query token and the j-th preceding token or compressed block. This technique allows each query head to adjust its total attention scores to be not equal to 1, and even to be near 0.

2.3.4. Efficiency Discussion

Due to the employment of hybrid CSA and HCA, together with low-precision computation and storage, the attention module of DeepSeek-V4 series achieves remarkable efficiency in both attention FLOPs and KV cache size, especially in long-context scenarios. First, we adopt a mixed storage format for KV entries: BF16 precision is used for the rotary positional embedding (RoPE) dimensions, while FP8 precision is applied to the remaining dimensions. This hybrid representation reduces the KV cache size by nearly half compared with pure BF16 storage. Second, attention computation within the lightning indexer is performed in FP4 precision, which accelerates the attention operation under extremely long contexts. Third, relative to DeepSeek-V3.2, a smaller attention top-k is chosen in DeepSeek-V4 series, thereby improving model efficiency on short- and medium-length texts. Finally, and most importantly, compressed attention and hybrid attention techniques substantially reduce both the KV cache size and the computational FLOPs.

Taking BF16 GQA8 (Ainslie et al., 2023) with a head dimension of 128 as the baseline — one of the common configurations of LLM attention — the KV cache size of DeepSeek-V4 series can be dramatically reduced to approximately 2% times of that baseline in the 1M-context setting.

Algorithm 1 Muon Optimizer for DeepSeek-V4

Require: Learning rate η, momentum μ, weight decay λ, update rescaling factor γ

for each training step t **do**
  for each logically independent weight $W \in \mathbb{R}^{n \times m}$ **do**
    $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$ ▷ Compute gradients
    $M_t = \mu M_{t-1} + G_t$ ▷ Accumulate momentum buffer
    $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$ ▷ Nesterov trick and hybrid Newton-Schulz
    $O_t = O'_t \cdot \sqrt{\max(n, m)} \cdot \gamma$ ▷ Rescale the update RMS
    $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ ▷ Perform weight decay and update
  end for
end for

Moreover, even when compared with DeepSeek-V3.2 (DeepSeek-AI, 2025) — already an efficient baseline — DeepSeek-V4 series still exhibits substantial advantages in efficiency. A comparison of their inference FLOPs and KV cache size is provided in the right part of Figure 1.

2.4. Muon Optimizer

We employ the Muon (Jordan et al., 2024; Liu et al., 2025) optimizer for the majority of modules in DeepSeek-V4 series due to its faster convergence and improved training stability. The full algorithm of our Muon optimization is summarized in Algorithm 1.

Basic Configurations. We maintain the AdamW (Loshchilov and Hutter, 2017) optimizer for the embedding module, the prediction head module, the static biases and gating factors of mHC modules, and the weights of all RMSNorm modules. All other modules are updated with Muon. Following Liu et al. (2025), we also apply weight decay to Muon parameters, use the Nesterov (Jordan et al., 2024; Nesterov, 1983) trick, and rescale the Root Mean Square (RMS) of the update matrix for reutilization of our AdamW hyper-parameters. Different from them, we use hybrid Newton-Schulz iterations for orthogonalization.

Hybrid Newton-Schulz Iterations. For a given matrix M, let its Singular Value Decomposition (SVD) be $M = U\Sigma V^T$. The Newton-Schulz iterations aim to approximately orthogonalize M to be UV^T. Usually, M will be first normalized as $M_0 = M/||M||_F$ to ensure its maximum singular value does not exceed 1. Then, each Newton-Schulz iteration performs the following operation:

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2 M_{k-1}. \tag{28}$$

Our hybrid Newton-Schulz performs 10 iterations over two distinct stages. During the first 8 steps, we use coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ to drive rapid convergence, bringing the singular values close to 1. In the final 2 steps, we switch to coefficients $(a, b, c) = (2, -1.5, 0.5)$, which stabilize the singular values precisely at 1.

Avoiding Exploding Attention Logits. The attention architecture of DeepSeek-V4 series allows us to directly apply RMSNorm on the attention queries and KV entries, which effectively prevents attention logits from exploding. Consequently, we do not employ the QK-Clip technique (Liu et al., 2025) in our Muon optimizer.